

Cryptography and Network Security Chapter 2 and 3

FIFTH EDITION

BY WILLIAM STALLINGS

Autokey Cipher

- An **autokey cipher**, or **self-synchronizing stream cipher**, is a cipher which incorporates the message into the key.
- In order to encrypt a letter, one locates the line starting with the letter to be encrypted, and the column beginning with the next letter of the key. The letter where the line and column cross is the ciphertext letter.

For example, if the plaintext is MEET ME FOR LUNCH and the key is RESTAURANT, the first letter of the ciphertext is D, because the M and R intersect at D. The second ciphertext character is I, where the E from MEET and the E from RESTAURANT intersect in the table.

more secure way to generate the key is to append the plaintext to it. This method makes frequency counting more difficult, as the key does not follow a simple repeating pattern:

a	b	c	d	e	f	g	h	i	j	k	l	m	n	o	p	q
r	s	t	u	v	w	x	y	z								
0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16
	17	18	19	20	21	22	23	24	25							

▶ Plaintext: MEETMEFORLUNCH

▶ Key: RESTAURANTMEET

▶ Ciphertext: DIWMMYWOEEGRGA

One-Time Pad

- In cryptography, the **one-time pad (OTP)** is an encryption algorithm in which the plaintext is combined with a secret random key or *pad*, which is used only once.
- The key is truly random, as large as the plaintext, never reused in whole or part, and kept secret.
- You have a message M , and you compute the ciphertext C by XORing the message with the OTP:
- $C = M \text{ xor } \text{OTP}$



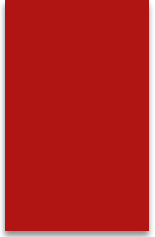
- ▶ now consider classical **transposition** or **permutation** ciphers
- ▶ these hide the message by rearranging the letter order
- ▶ without altering the actual letters used

Transposition Ciphers

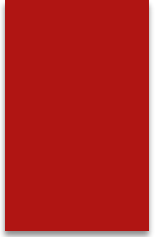
- A cipher, or a component of a more complicated cipher, that involves the symbol at each place in the plaintext being moved to a newly decided (and often different) place, i.e. within each block of plaintext the positional indices of the symbol places are changed according to some plan.

Product Ciphers

- ciphers using substitutions or transpositions are not secure because of language characteristics
- hence consider using several ciphers in succession to make harder, but:
 - two substitutions make a more complex substitution
 - two transpositions make more complex transposition
 - but a substitution followed by a transposition makes a new much harder cipher
- this is bridge from classical to modern ciphers



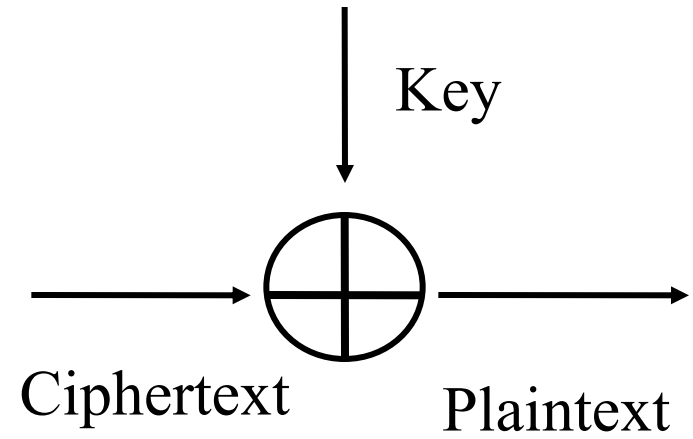
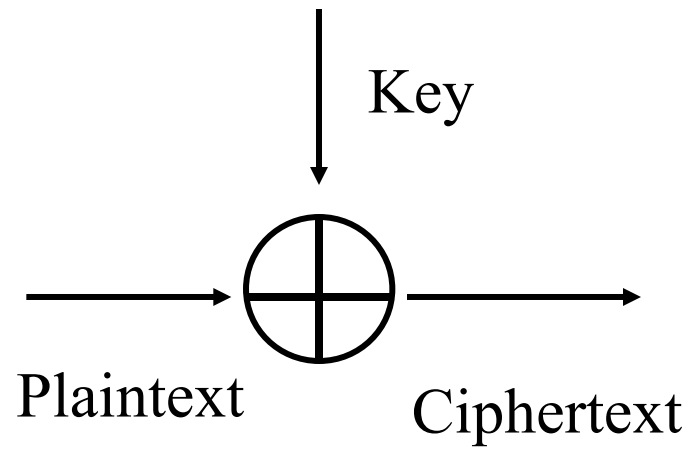
Chapter 3 – “Block Ciphers and the Data Encryption Standard”

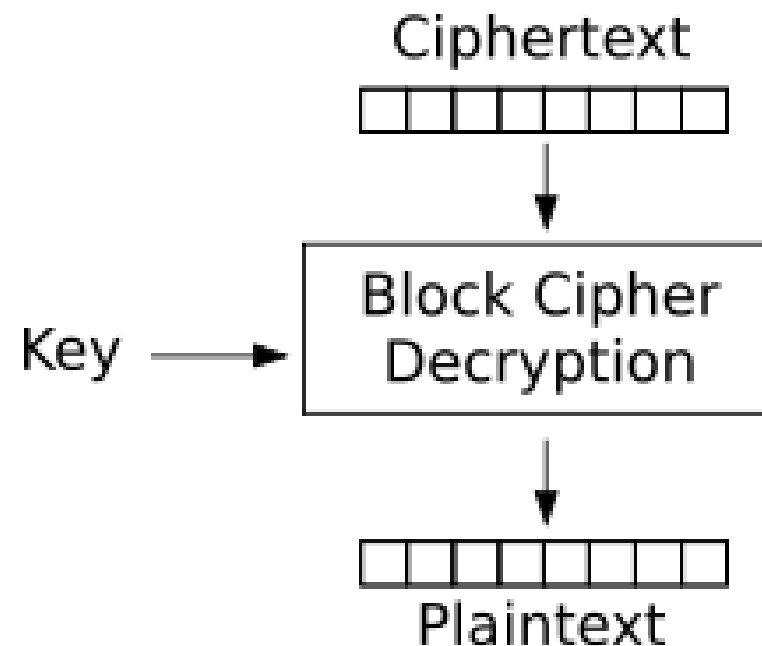
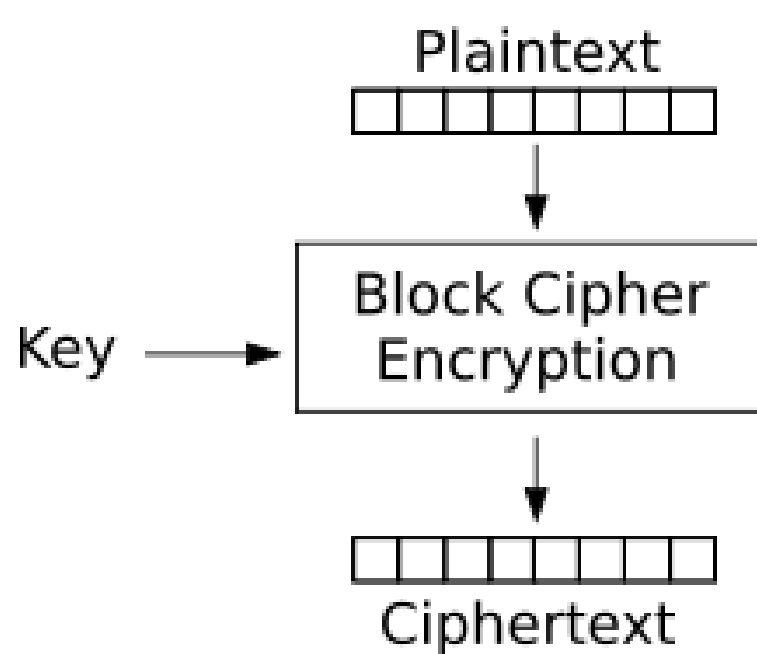


Block Ciphers and the Data Encryption Standard

Block vs Stream Ciphers

- ▶ block ciphers process messages in blocks, each of which is then en/decrypted
- ▶ like a substitution on very big characters
 - ▶ 64-bits or more
- ▶ stream ciphers process messages a bit or byte at a time when en/decrypting
- ▶ many current ciphers are block ciphers
- ▶ broader range of applications





Block Cipher Principles

- ▶ most symmetric block ciphers are based on a **Feistel Cipher Structure**
- ▶ needed since must be able to **decrypt** ciphertext to recover messages efficiently
- ▶ block ciphers look like an extremely large substitution
- ▶ would need table of 2^{64} entries for a 64-bit block
- ▶ instead create from smaller building blocks
- ▶ using idea of a product cipher

Claude Shannon and Substitution-Permutation Ciphers

- ▶ Claude Shannon introduced idea of substitution-permutation (S-P) networks in 1949 paper
- ▶ form basis of modern block ciphers
- ▶ S-P nets are based on the two primitive cryptographic operations seen before:
 - ▶ *substitution* (S-box)
 - ▶ *permutation* (P-box)
- ▶ provide *confusion* & *diffusion* of message & key

Confusion and Diffusion

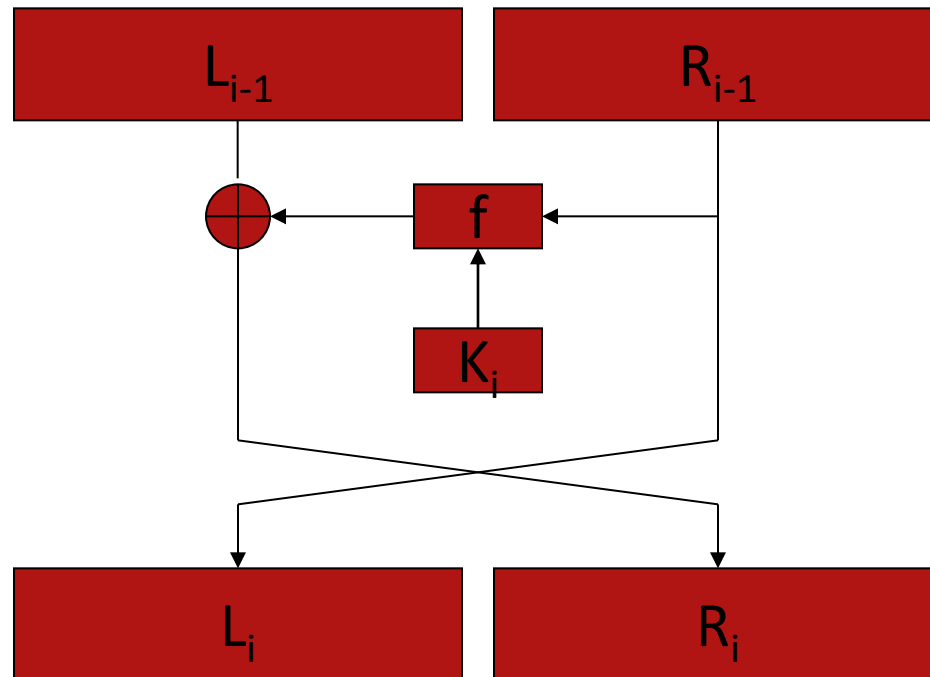
- ▶ cipher needs to completely obscure statistical properties of original message
- ▶ a one-time pad does this
- ▶ more practically Shannon suggested combining S & P elements to obtain:
- ▶ **diffusion** – dissipates statistical structure of plaintext over bulk of ciphertext
- ▶ **confusion** – makes relationship between ciphertext and key as complex as possible

Feistel Cipher Structure

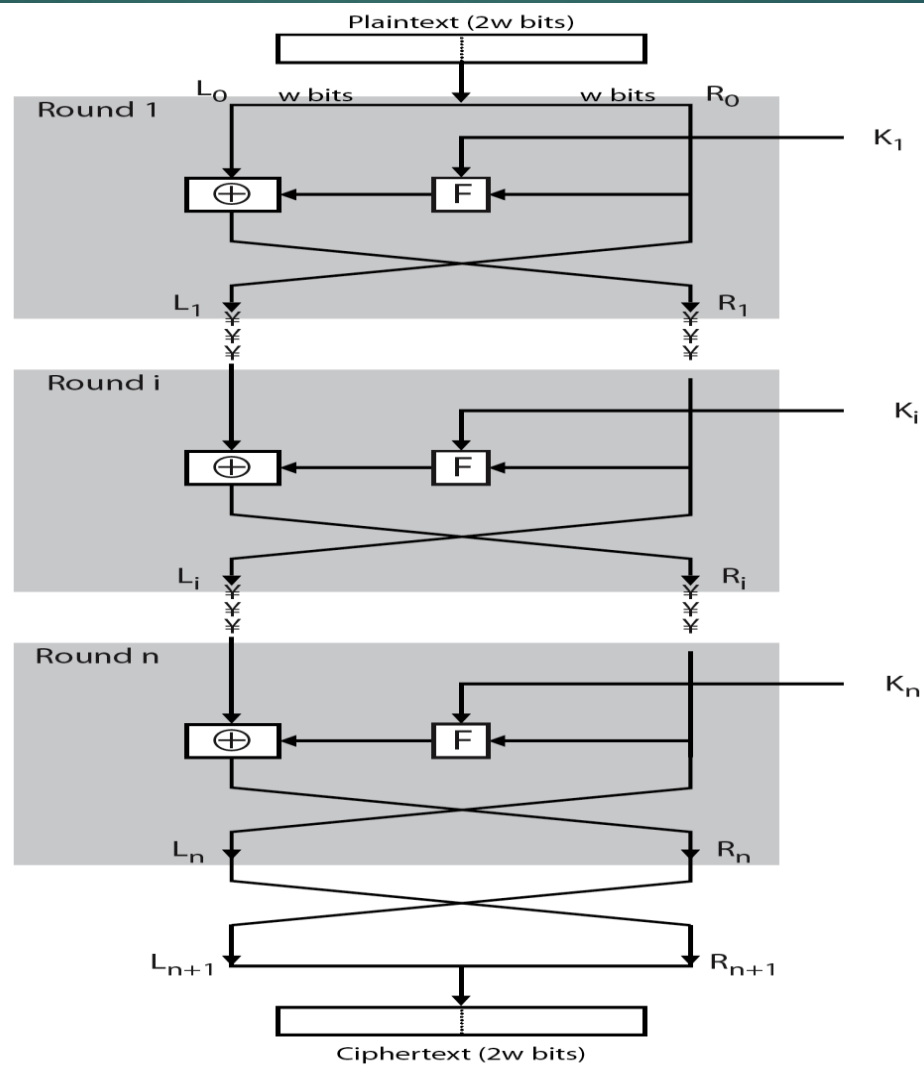
- ▶ Horst Feistel devised the **feistel cipher**
 - ▶ based on concept of invertible product cipher
- ▶ partitions input block into two halves
 - ▶ process through multiple rounds which
 - ▶ perform a substitution on left data half
 - ▶ based on round function of right half & subkey
 - ▶ then have permutation swapping halves
- ▶ implements Shannon's S-P net concept

Feistel Network

- ▶ iterated cipher mapping (L_0, R_0) to (R_r, L_r) through r -round process, $(L_{i-1}, R_{i-1}) \rightarrow_{K_i} (L_i, R_i)$ as follows
- ▶ $L_i = R_{i-1}$, $R_i = L_{i-1} \oplus f(R_{i-1}, K_i)$, K_i is derived from K



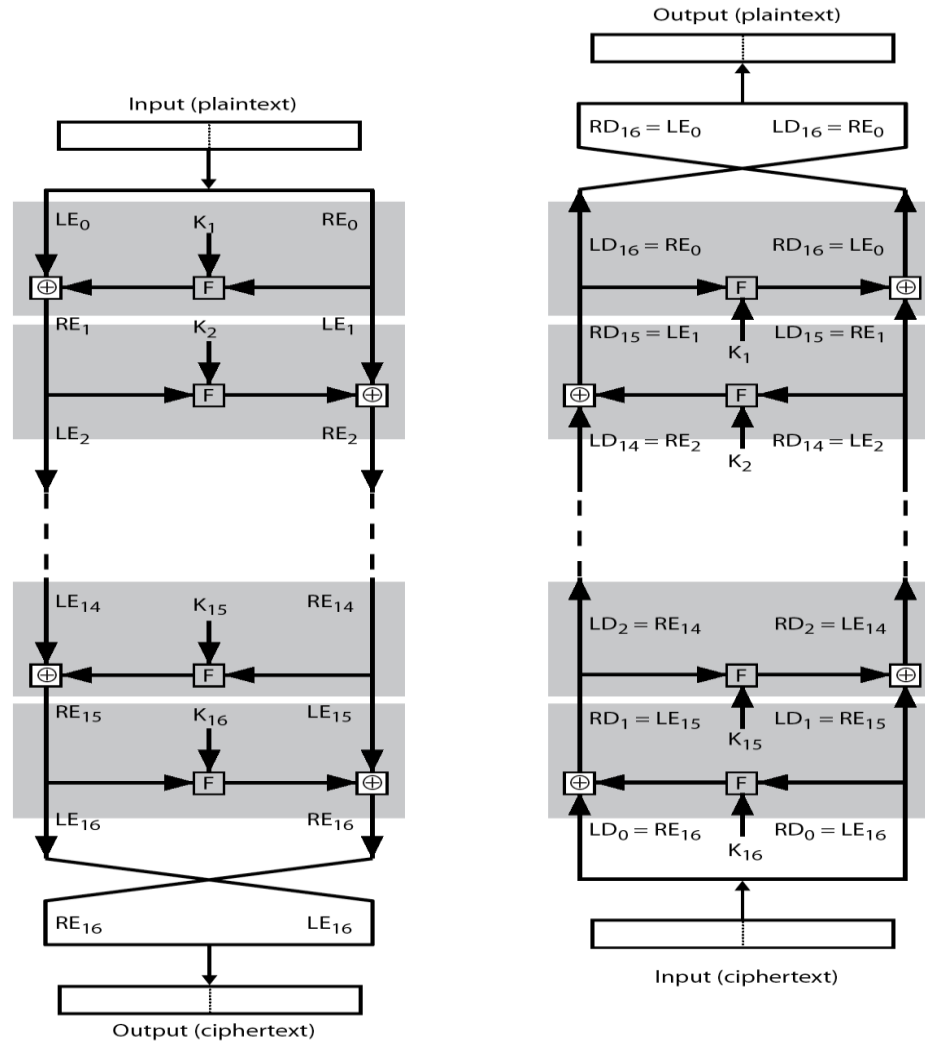
Feistel Cipher Structure



Feistel Cipher Structure

- ▶ **Block size:** larger block sizes mean greater security
- ▶ **Key Size:** larger key size means greater security
- ▶ **Number of rounds:** multiple rounds offer increasing security
- ▶ **Subkey generation algorithm:** greater complexity will lead to greater difficulty of cryptanalysis.
- ▶ **Fast software encryption/decryption:** the speed of execution of the algorithm becomes a concern

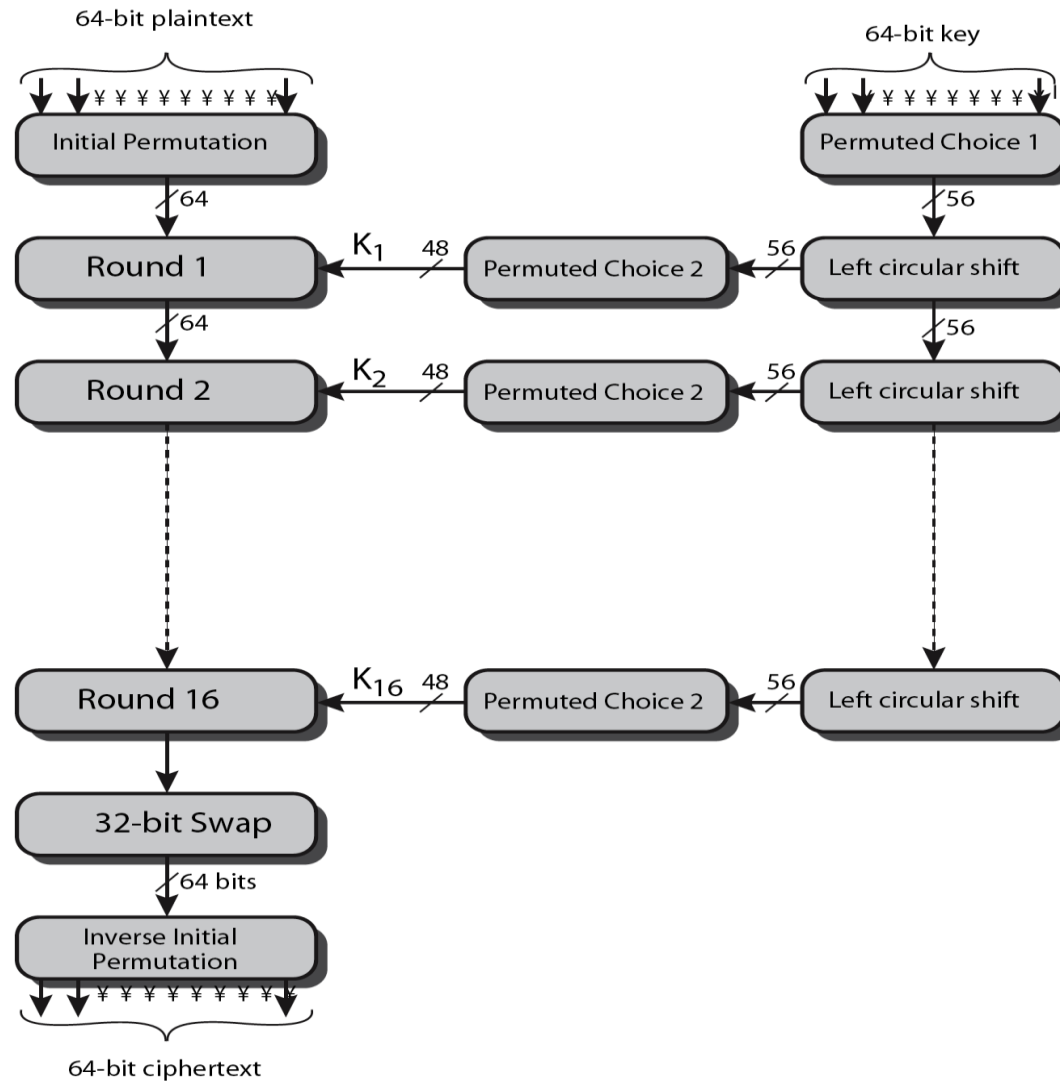
Feistel Cipher Decryption



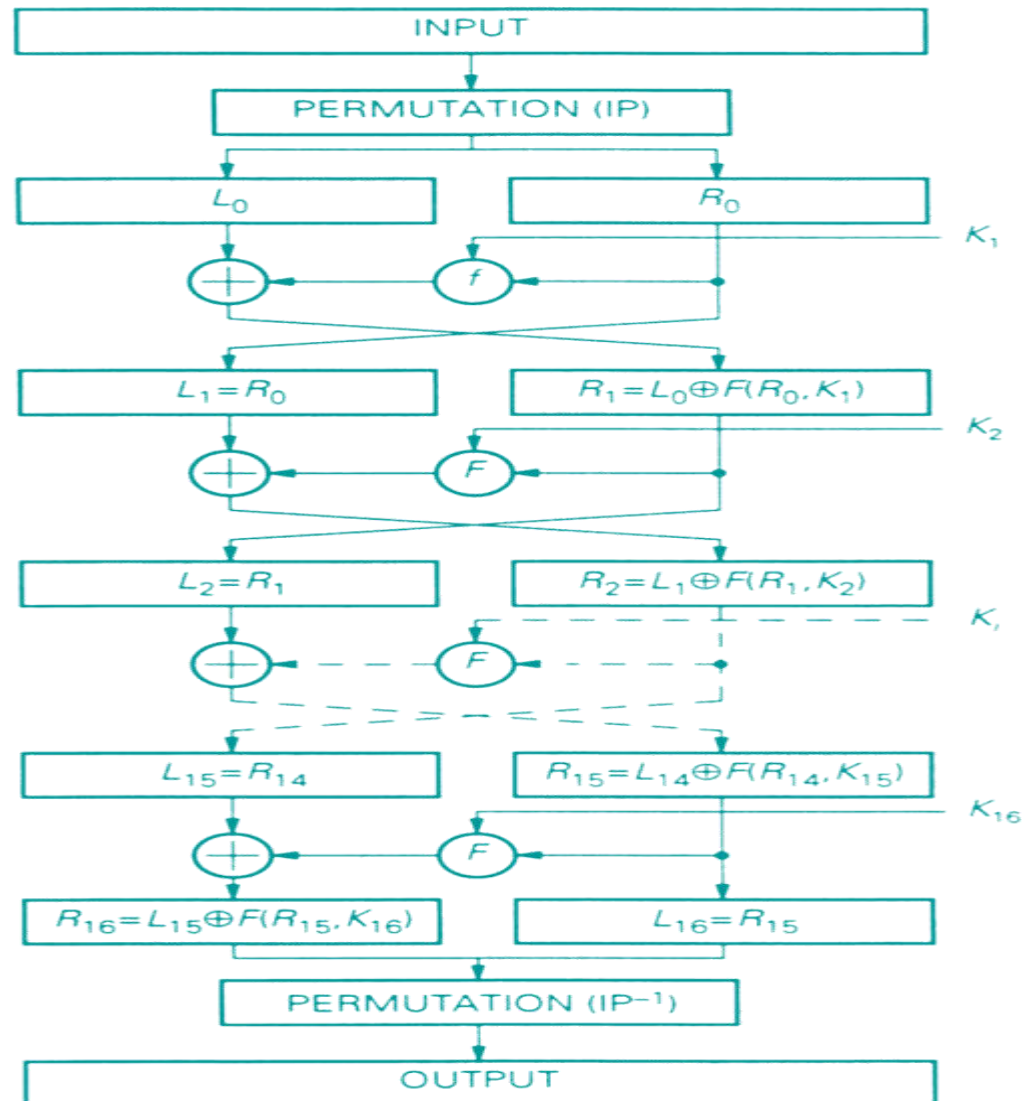
Data Encryption Standard (DES)

- ▶ most widely used block cipher in world
- ▶ adopted in 1977 by NBS (now NIST)
 - ▶ as FIPS PUB 46
- ▶ encrypts 64-bit data using 56-bit key
- ▶ has widespread use
- ▶ has been considerable controversy over its security

DES Encryption Overview



DES



DES Round Structure

- ▶ uses two 32-bit L & R halves
- ▶ as for any Feistel cipher can describe as:

$$L_i = R_{i-1}$$

$$R_i = L_{i-1} \oplus F(R_{i-1}, K_i)$$

- ▶ F takes 32-bit R half and 48-bit subkey:
 - ▶ expands R to 48-bits using perm E
 - ▶ adds to subkey using XOR
 - ▶ passes through 8 S-boxes to get 32-bit result
 - ▶ finally permutes using 32-bit perm P

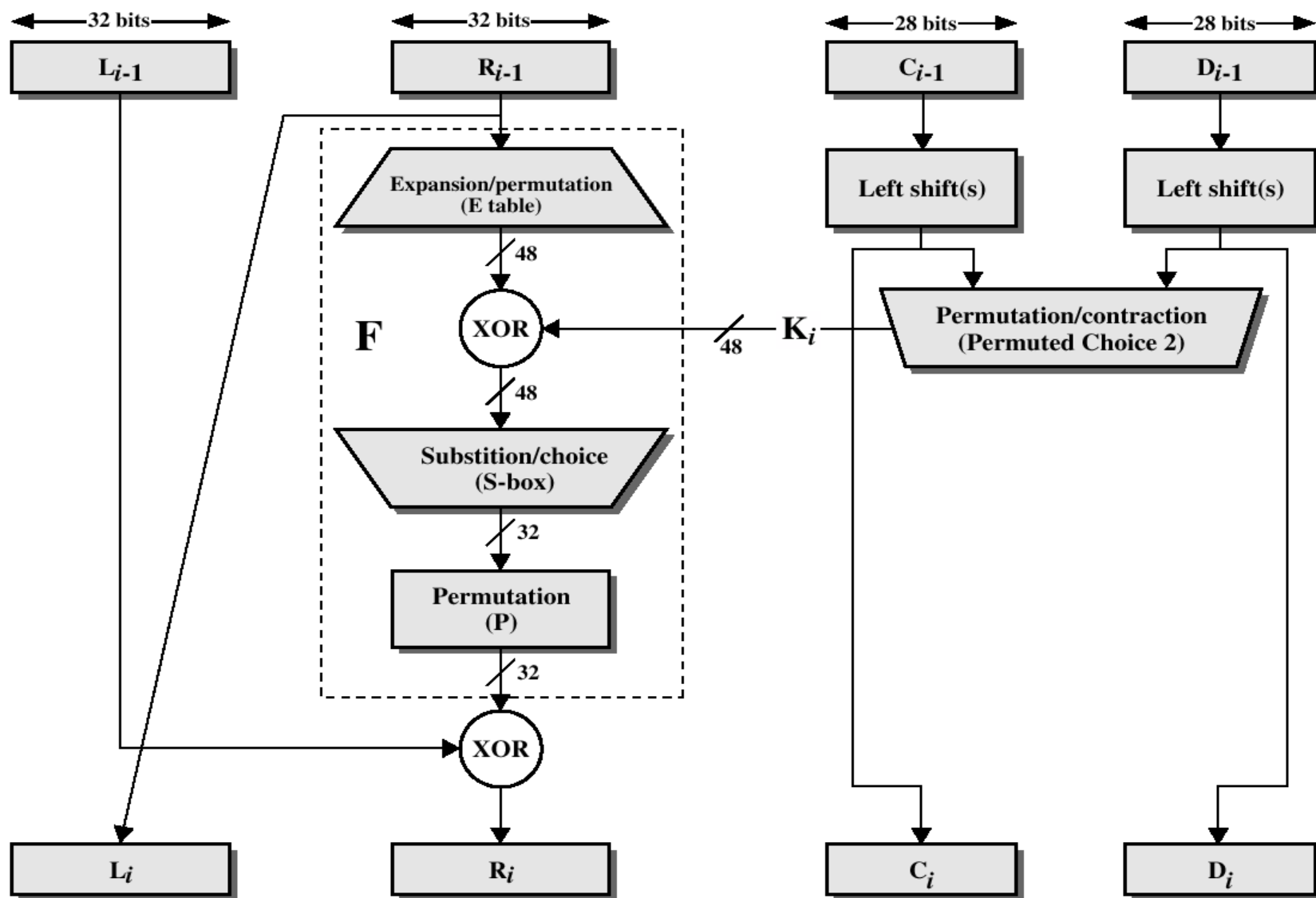


Figure 2.4 Single Round of DES Algorithm

Initial Permutation IP

- ▶ first step of the data computation
- ▶ IP reorders the input data bits
- ▶ even bits to LH half, odd bits to RH half
- ▶ The values in each matrix identify where each bit of the input message is mapped to in the output message. For example, The matrix for *IP* shows that the 58th bit from the input gets mapped to the first bit of the output; the 50th of the input maps to the second of the output, and so on.



IP								IP ⁻¹							
58	50	42	34	26	18	10	2	40	8	48	16	56	24	64	32
60	52	44	36	28	20	12	4	39	7	47	15	55	23	63	31
62	54	46	38	30	22	14	6	38	6	46	14	54	22	62	30
64	56	48	40	32	24	16	8	37	5	45	13	53	21	61	29
57	49	41	33	25	17	9	1	36	4	44	12	52	20	60	28
59	51	43	35	27	19	11	3	35	3	43	11	51	19	59	27
61	53	45	37	29	21	13	5	34	2	42	10	50	18	58	26
63	55	47	39	31	23	15	7	33	1	41	9	49	17	57	25

Table 2. Initial and final permutation matrices for DES.

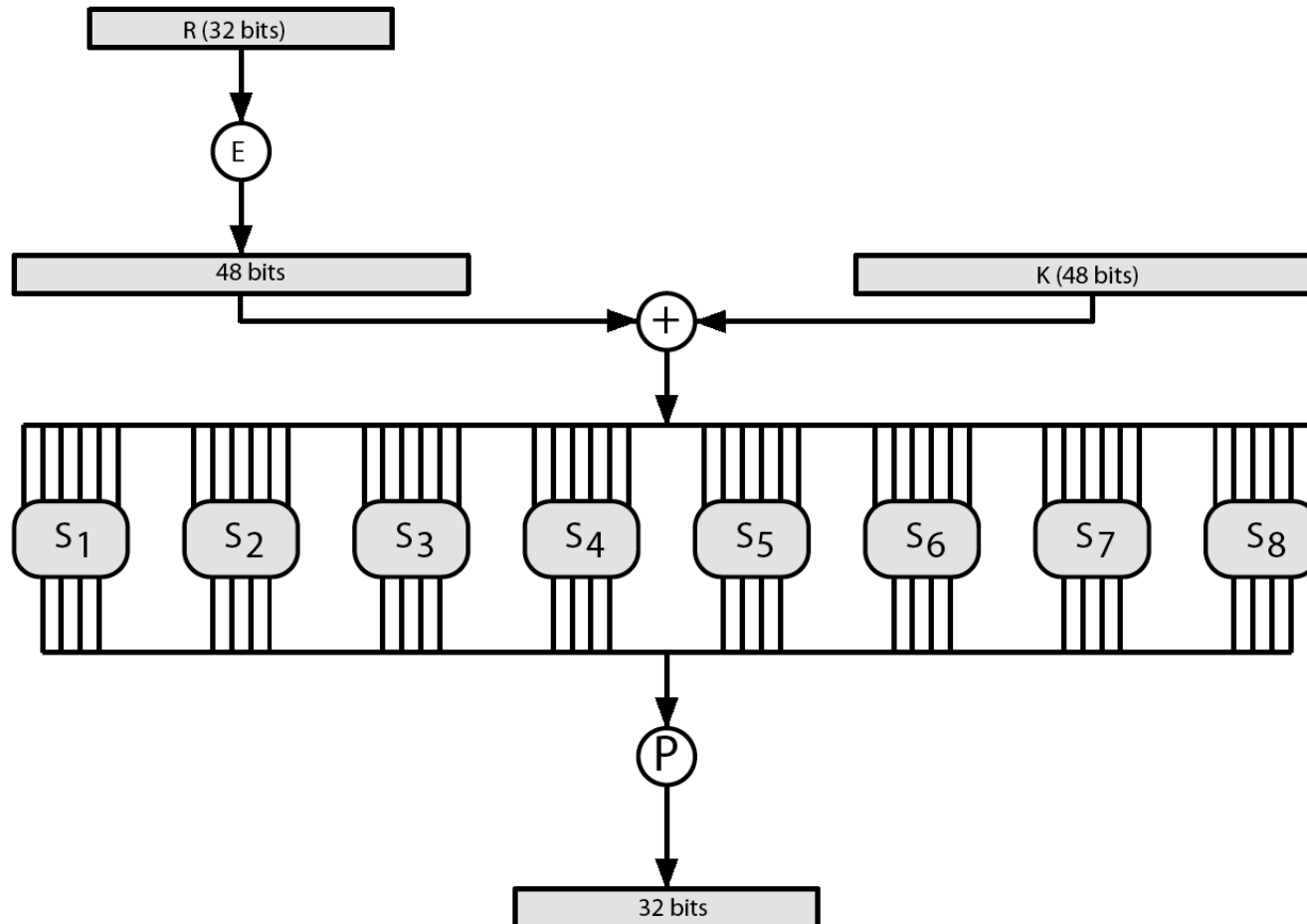
- 
- ▶ Expands the 32 bit data to 48 bits
 - ▶ $\text{Result}(i) = \text{input}(\text{array}(i))$
 - ▶ The expansion table defines a permutation plus an expansion that involves duplication of 16 of the bits.

32	1	2	3	4	5
4	5	6	7	8	9
8	9	10	11	12	13
12	13	14	15	16	17
16	17	18	19	20	21
20	21	22	23	24	25
24	25	26	27	28	29
28	29	30	31	32	1

- 
- ▶ The resulting 48 bits are XORed with key.
 - ▶ This 48 bit result passes through a substitution function comprising 8 S-boxes.
 - ▶ Which each map 6 input bits to 4 output bits.

 - ▶ Given 6-bits $B=b_1b_2b_3b_4b_5b_6$,
 - ▶ Row $r=b_1b_6$
 - ▶ Column $c=b_2b_3b_4b_5$
 - ▶ $S(B)=S(r,c)$ written in binary of length 4

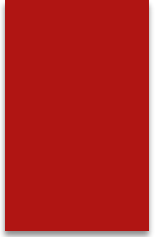
DES Round Structure



S-Box

- ▶ 6 bit input, 4 bit output
- ▶ $27 = \mathbf{011011} = (\mathbf{01})$
(1101)
- ▶ S_1 -Box output for 27 = 5

	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
0	14	4	13	1	2	15	11	8	3	10	6	12	5	9	0	7
1	0	15	7	4	14	2	13	1	10	6	12	11	9	5	3	8
2	4	1	14	8	13	6	2	11	15	12	9	7	3	10	5	0
3	15	12	8	2	4	9	1	7	5	11	3	14	10	0	6	13

- 
- The permutation after each round

16	7	20	21
29	12	28	17
1	15	23	26
5	18	31	10
2	8	24	14
32	27	3	9
19	13	30	6
22	11	4	25

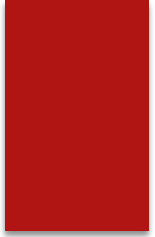
DES Key Schedule

- ▶ The 64 –bit key input is first processed by permuted choice One.
- ▶ The resulting 56-bit key is then treated as two 28-bit quantities C and D.
- ▶ In each round forms subkeys
- ▶ rotating **each half** separately either 1 or 2 places depending on the **key rotation schedule K**
 - ▶ selecting 24-bits from each half & permuting them by PC2 for use in round function F
- ▶ note practical use issues in h/w vs s/w



57	49	41	33	25	17	9
1	58	50	42	34	26	18
10	2	59	51	43	35	27
19	11	3	60	52	44	36
63	55	47	39	31	23	15
7	62	54	47	38	30	22
14	6	61	53	45	37	29
21	13	5	28	20	12	4

Permutation table P1

- 
- After this permutation, the key is split into two halves, **C** and **D**. After each round, each half is independently shifted to the left by either one or two bits, depending on which round is executing. The shift is rotational, so that bits that get shifted off of one end get placed back on the other end.

Round	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16
Shifts	1	1	2	2	2	2	2	2	1	2	2	2	2	2	2	1

Table. Key Shifting Factors for each round of DES.

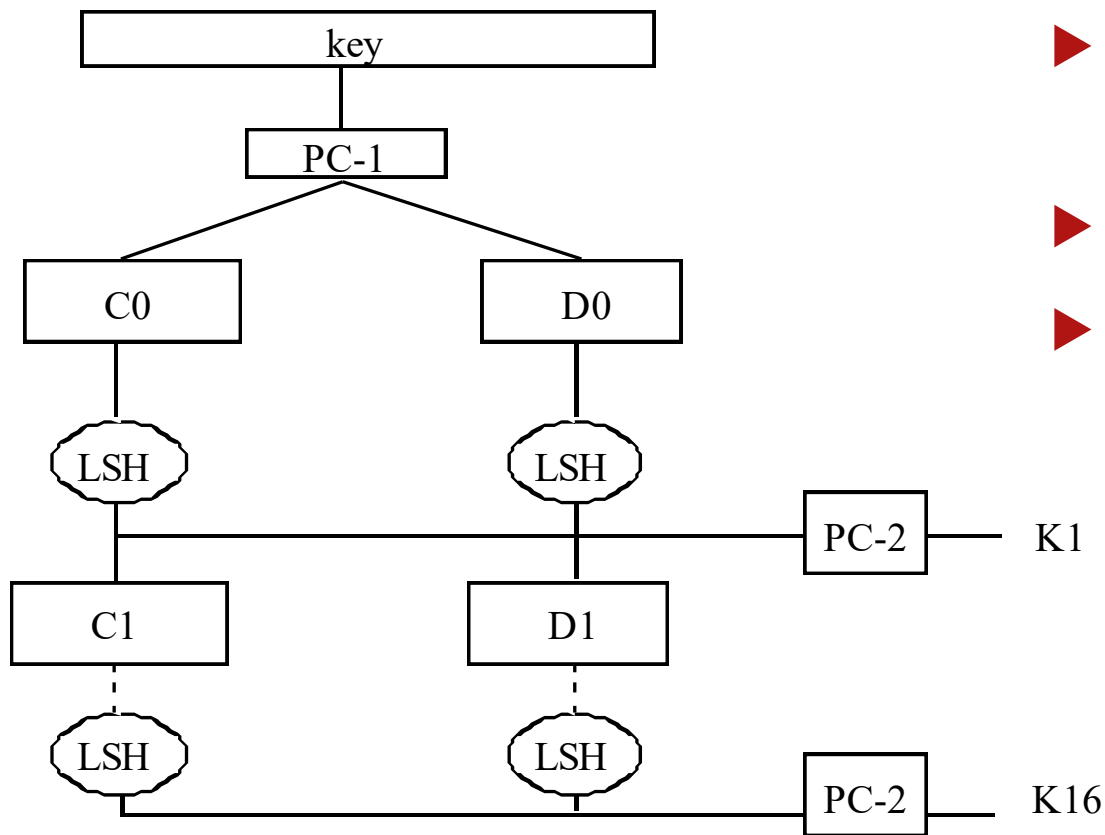
Finally, the subkey function is used to convert the key into a 48 bit block, to be used in the actual encryption. Again, this is expressed in matrix form, as shown below.

Subkey Permutation

14	17	11	24	1	5
3	28	15	6	21	10
23	19	12	4	26	8
16	7	27	20	13	2
41	52	31	37	47	55
30	40	51	45	33	48
44	49	39	56	34	53
46	42	50	36	29	32

DES Decryption

- ▶ decrypt must unwind steps of data computation
- ▶ with Feistel design, do encryption steps again using subkeys in reverse order (SK16 ... SK1)
 - ▶ IP undoes final FP step of encryption
 - ▶ 1st round with SK16 undoes 16th encrypt round
 - ▶
 - ▶ 16th round with SK1 undoes 1st encrypt round
 - ▶ then final FP undoes initial encryption IP
 - ▶ thus recovering original data value



- ▶ Round keys are 48 bits each
- ▶ LSH == left shift
- ▶ PC-1, PC-2 == permutations

Avalanche Effect

- ▶ A desirable property of any encryption algorithm is that a small change in either the plaintext or the key should produce a significant change in the ciphertext.
- ▶ In particular, a change in one bit of the plaintext or one bit of the key should produce a change in many bits of the ciphertext.
- ▶ DES exhibits strong avalanche

Strength of DES – Key Size

- ▶ 56-bit keys have $2^{56} = 7.2 \times 10^{16}$ values
- ▶ brute force search looks hard
- ▶ recent advances have shown is possible
 - ▶ in 1997 on Internet in a few months
 - ▶ in 1998 on dedicated h/w Electronic Frontier Foundation (EFF) in a few days
 - ▶ in 1999 above combined in 22hrs!
- ▶ still must be able to recognize plaintext
- ▶ must now consider alternatives to DES